CS641: Modern Cryptology - Assignment 1

B Mukund Advaith
240253
bmukund24@iitk.ac.in

June 12, 2026

Part 1 Questions

1. One-time pad (OTP)

(a) Recall, we say that a cryptosystem is perfectly secret if

$$Pr(P = p|C = c) = Pr(P = p)$$

for all possible plaintexts p and all possible ciphertexts c. In other words, by observing the ciphertext, the knowledge you learned about a given plaintext is ZERO.

Express $Pr(P = p|C = c)$ using Bayes's theorem, we get:

$$Pr(P = p|C = c) = \frac{Pr(C = c|P = p) \cdot Pr(P = p)}{Pr(C = c)}$$

Recall that for a uniformly distributed key $k \in \{0,1\}^n$, there are 2^n possible keys, each chosen with a probability of $\frac{1}{2^n}$. For a fixed plaintext p and ciphertext c, there is exactly one unique key k that satisfies $c = p \oplus k$ (specifically, $k = p \oplus c$).

Therefore:

$$Pr(C = c|P = p) = Pr(Enc(k, p) = c) = Pr(k = p \oplus c) = \frac{1}{2^n}$$

Also, by using law of total probability,

$$Pr(C = c) = \sum_{p \in \{0,1\}^n} Pr(C = c|P = p) \cdot Pr(P = p)$$

$$Pr(C = c) = \sum_{p \in \{0,1\}^n} \frac{1}{2^n} \cdot Pr(P = p) = \frac{1}{2^n} \sum_{p \in \{0,1\}^n} Pr(P = p)$$

Since the sum of the probabilities of all possible plaintexts equals 1, this simplifies to:

$$Pr(C = c) = \frac{1}{2^n}$$

Substituting our derived values for $Pr(C = c|P = p)$ and $Pr(C = c)$ back into the Bayes's theorem equation:

$$Pr(P = p|C = c) = \frac{\frac{1}{2^n} \cdot Pr(P = p)}{\frac{1}{2^n}} = Pr(P = p)$$

Hence, proved that the one-time pad encryption scheme is perfectly secure.

(b) Given,

$$c_1 = 00100010110101011010010011000101111010101011111$$
$$c_2 = 00110101110101011010010011011000111010001001110$$

Also given,

$$p_1 = secret\ or\ public$$
$$p_2 = encode\ or\ decode$$

In two-time pad, the same key is used for both plaintexts, therefore

$$c_1 = p_1 \oplus k$$

$$c_2 = p_2 \oplus k$$

Combining these two by XORing, we get

$$c_1 \oplus c_2 = p_1 \oplus p_2 \oplus k \oplus k = p_1 \oplus p_2$$

Thus, we can do $c_1 \oplus c_2$ and check if it matches with any of the 4 different cases of $p_1 \oplus p_2$.

$$c_1 \oplus c_2 = 00010111000000000000000000011101000000100010001$$

$$secret \oplus encode = 00010110000010110000000000011101000000100010001$$
$$secret \oplus decode = 00010111000000000000000000011101000000100010001$$
$$public \oplus encode = 00010101000110110000000100000011000011010000110$$
$$public \oplus decode = 00010100000100000000001000000110000110100000110$$

Hence, we can clearly see that $p_1, p_2 = secret, decode$.

(c) 1. To decrypt the ciphertext c, we need to isolate the plaintext p from the encryption equation:

$$c = p \cdot k \pmod{n}$$

Since n is a prime number and the key k is drawn from $\mathbb{Z}_n^+ = \{1, 2, \ldots, n-1\}$, the key k and n are co-prime ($\gcd(k, n) = 1$). This guarantees that k has a unique modular multiplicative inverse modulo n, which we denote as $k^{-1} \pmod{n}$.

Multiplying both sides of the encryption equation by k^{-1}:

$$c \cdot k^{-1} = p \cdot k \cdot k^{-1} \pmod{n}$$

$$p = c \cdot k^{-1} \pmod{n}$$

Decryption Function:

$$p = \text{Dec}(k, c) = c \cdot k^{-1} \pmod{n}$$

2. No, this scheme is not perfectly secure. Perfect secrecy requires that observing the ciphertext reveals absolutely no information about the plaintext. Mathematically,

$$Pr(P = p|C = c) = Pr(P = p)$$

However, in this multiplicative scheme, consider what happens when the plaintext $p = 0$:

$$c = 0 \cdot k \ (mod \ n) = 0$$

Thus, when c = 0, we can say with 100% accuracy that p = 0. Similarly, when $c \neq 0$, we can say with 100% accuracy that $p \neq 0$. Because the ciphertext alters the adversary's knowledge of the probability distribution of the plaintexts, perfect secrecy is broken.

We can also argue that a necessary condition for perfect secrecy according Shannon is that the size of key space $|\mathcal{K}|$, should be greater than or equal to the size of message space $|\mathcal{M}|$. Here,

- The message/plaintext space is $\mathbb{Z}_n = \{0, 1, 2, ..., n-1\}$, so $|\mathcal{M}| = n$.
- The key space is $\mathbb{Z}_n^+ = \{1, 2, ..., n-1\}$, so $|\mathcal{K}| = n - 1$.

Because $n - 1 < n$, there are simply not enough keys available to perfectly hide all possible plaintexts. Therefore, it cannot be perfectly secure.

(d) Given, $length \ of \ key, \ |k| = 28$.

Also given,
$c = 0x221C05471C0E00551B09151D4F171C550B4F164F1301011C1D000E04$
$6E20646F20666F7220796F752C2061736B207768617420796F752063$
$616E20646F20666F7220796F757220636F756E7472792E204A464B$
As ASCII encoding is used for plaintext, $length \ of \ plaintext, \ |c| = 83$.

For a one-time pad, it is required that the length of key and plaintext be the same. However, this is not the case here, creating a vulnerability in the scheme.
Using a 28-character key on a 83-character plaintext will only encrypt the first 28 characters of plaintext, using the XOR operation. The latter 55 characters of the plaintext will remain unencrypted.
So, in the ciphertext, only the first 28 characters are encrypted, while the remaining 55 characters reveal a part of the plaintext.

Converting the hex values of the last 55 characters of the ciphertext to their ASCII translation, we get "n do for you, ask what you can do for your country. JFK". This is the last 55 characters of the plaintext.
This is part of a popular quote by former President of the United States of America, John F. Kennedy. The complete quote is "Ask not what your country can do for you, ask what you can do for your country."

Therefore, the first 28 characters of the plaintext is "Ask not what your country ca". The complete plaintext is "Ask not what your country can do for you, ask what you can do for your country. JFK"

The key can be recovered using the following formula, $k = c \oplus p$, i.e. XORing the first 28 characters of ciphertext and plaintext.

c_i (hex)	p_i (char)	p_i (hex)	$k_i = c_i \oplus p_i$ (hex)	k_i (char)
22	A	41	63	c
1C	s	73	6F	o
05	k	6B	6E	n
47		20	67	g
1C	n	6E	72	r
0E	o	6F	61	a
00	t	74	74	t
55		20	75	u
1B	w	77	6C	l
09	h	68	61	a
15	a	61	74	t
1D	t	74	69	i
4F		20	6F	o
17	y	79	6E	n
1C	o	6F	73	s
55	u	75	20	
0B	r	72	79	y
4F		20	6F	o
16	c	63	75	u
4F	o	6F	20	
13	u	75	66	f
01	n	6E	6F	o
01	t	74	75	u
1C	r	72	6E	n
1D	y	79	64	d
00		20	20	
0E	c	63	6D	m
04	a	61	65	e

The key is found to be, $k =$ "congratulations you found me".

2. Affine Ciphers

(a) To decrypt the ciphertext c, we need to isolate the plaintext p from the encryption equation.

$$c = E_{(a,b)}(p) = a \cdot (p + b) \pmod{27}$$

Multiplying both sides of the encryption equation by a^{-1}:

$$a^{-1} \cdot c = a^{-1} \cdot a \cdot (p + b) \pmod{27}$$

$$a^{-1} \cdot c = (p + b) \pmod{27}$$

Subtract b from both sides:

$$p = (a^{-1} \cdot c) - b \pmod{27}$$

Decryption Function:

$$p = D_{(a,b)}(c) = (a^{-1} \cdot c) - b \pmod{27}$$

Important point: For a^{-1} to exist, a must be coprime to 27, i.e. $gcd(a, 27) = 1$. This means that a should belong to $\mathbb{Z}_{27}^{+}$.

(b) Let's first find the total number of keys in the key space, and subtract the number of trivial keys, to get the number of non-trivial keys.

- Total number of keys in key space:
  - a can take values from $\mathbb{Z}_{27}^{+}$. Therefore, $n(\mathbb{Z}_{27}^{+}) = \phi(27) = 18$.
  - b can take values from $\mathbb{Z}_{27}$. Therefore, $n(\mathbb{Z}_{27}) = 27$.

  Therefore, total valid keys $= 18 \cdot 27 = 486$
- Number of trivial keys: For a key to be trivial, the condition $a(p + b) = p \pmod{27}$ must hold true for every $p \in \mathbb{Z}_{27}$. Let's test specific values of p to find the conditions for a and b:
  - Put $p = 0$,
  $$a \cdot (0 + b) = 0 \pmod{27} \implies ab = 0 \pmod{27}$$
  - Put $p = 1$,
  $$a \cdot (1 + b) = 1 \pmod{27} \implies a + ab = 1 \pmod{27}$$

  Substituting $ab = 0$ into the second equation yields:

  $$a + 0 = 1 \pmod{27} \implies a = 1 \pmod{27}$$

  Now substitute $a = 1$ back into the first condition ($ab = 0$):

  $$1 \cdot b = 0 \implies b = 0$$

  Thus, the only choice of parameters that yields $c = p$ for all p is the single key pair $(a, b) = (1, 0)$. Therefore, number of trivial keys = 1.
- Number of non-trivial keys: Total valid keys - Number of trivial keys.

  Therefore, number of non-trivial keys = 486 - 1 = 485.

(c) We can determine the key from the helper by using a similar method to how we obtained the trivial key, i.e. by putting $p_1 = 0$ and $p_2 = 1$.

- Put $p_1 = 0$, let the obtained ciphertext be c_1, then:

$$c_1 = a \cdot (0 + b) = ab \pmod{27}$$

- Put $p_2 = 1$, let the obtained ciphertext be c_2, then:

$$c_2 = a \cdot (1 + b) = a + ab \pmod{27}$$

Substituting $ab = c_1$ into the second equation yields:

$$a + c_1 = c_2 \pmod{27} \implies a = c_2 - c_1 \pmod{27}$$

Now substitute $a = c_2 - c_1$ back into the first condition ($ab = c_1$):

$$(c_2 - c_1) \cdot b = c_1 \implies b = c_1 \cdot (c_2 - c_1)^{-1} \pmod{27}$$

Hence, the key (a,b) has been discovered.

3. Cryptanalysis of Monoalphabetic Cipher

Instead of using brute force attack on a simple monoalphabetic cipher to find plaintext, we can use language statistics to smartly decrypt the ciphertext and reveal the plaintext.

Here are the rules/steps that were followed to decrypt the ciphertext:

- Find the frequencies of each character in the ciphertext, and keep it in hand while decyrpting.
- The most frequent character in the ciphertext will map to the 'SPACE' character in plaintext, as it is the most used character in the English language, as it is used to separate words. Changing the most frequent character to space will also now give an idea of the word structure of the plaintext.
- The 2nd, 3rd and 4th most frequent characters will map to the 1st, 2nd and 3rd most frequent alphabets in the English language: 'e', 't', and 'a'.
- The most frequent 3-letter word in the English language is 'the'. Find the most frequent 3-letter string in the ciphertext and map it to 'the'. This is a way of checking the mappings of 't' and 'e', while also giving the mapping of 'h'.
- The most frequent character to precede a 'SPACE' character is the '.' (period) and then the ',' (comma), due to semantic rules usually followed in English. As such, find the frequency of characters preceding a 'SPACE' and map the top 2 to '.' and ',' respectively.
- The 4th, 5th, 6th and 7th frequent alphabets in the English language are 'o', 'i', 'n', and 's', but as their frequencies are relatively very close to each other (compared to the big gaps in the first 3 frequent letters), these cannot be directly mapped to the 5th-8th most frequent characters in the ciphertext.

  Eg. 'o' could be mapped to the 7th most frequent character, 'i' to the 8th, 'n' to the 6th and 's' to the 5th.

  Look for words in the partially decrypted text, such that they mostly contain decrypted letters (from previous steps), and the remaining encrypted letters of the word are one of the characters from 5th-8th most frequent characters in ciphertext. Put 'o', 'i' ,'n' or 's' and check if they make meaningful words.

  Eg. Blue letters are letters decrypted from previous steps, red letters are encrypted letters.

  thek: Here, k will map to 'n', as 'then' is a meaningful word, while 'theo' ,'thei' , and 'thes' are not.

  keak: Here, k will map to 's', as 'seas' is a meaningful word, while 'oeao' ,'ieai' , and 'nean' are not.

- Above steps would have decrypted the bulk of the ciphertext. Now, look through the partially decrypted text and try to match the partially decrypted words to known words. Use 2-3 words to confirm the mapping.

  Eg. Blue letters are letters decrypted from previous steps, red letters are encrypted letters.

  att, tittte, vesset: Here, t will map to 'l', making the words 'all', 'little' and 'vessel'.

  og, ogg: Here, g will map to 'f', making the words 'of' and 'off'.

  This is not a step which is only applied at the end, but is a step that is applied throughout the process whenever any recognisable words are seen.

Some other facts to keep in mind while decrypting:

- Single letter words can only be mapped to I or a.
- Some high frequency two letter words are: of, to, in, it, is, as, at, be, we, he etc.
- The most frequent three letter words after 'the' are: and, for, are, but, not, you etc.
- Use other grammatical rules like tenses (-ed and -ing), active-passive voice, direct-indirect speech and subject-verb concord.

Here is the mapping of the letters in given ciphertext to plaintext.

Cipher	Plain	Cipher	Plain	Cipher	Plain
a	o	k	s	u	n
b	a	l	x	v	d
c	space	m	i	w	m
d	t	n	j/z	x	.
e	v	o	u	y	c
f	,	p	e	z	q
g	f	q	k	space	w
h	g	r	y	.	p
i	j/z	s	h	,	r
j	b	t	l	—	—

The letters 'j' and 'z' never appear in the plaintext, so we are unsure of their mapping against 'i' and 'n', which never appear in the ciphertext.

Here is the recovered plaintext:

most annoying circumstances have brought you into the presence of a man who has broken all the ties of humanity. you have come to trouble my existence." "unintentionally!" said i. "unintentionally?" replied the stranger, raising his voice a little; "was it unintentionally that the abraham lincoln pursued me all over the seas? was it unintentionally that you took passage in this frigate? was it unintentionally that your cannon balls rebounded off the plating of my vessel? was it unintentionally that mr. ned land struck me with his harpoon?" i detected a restrained irritation in these words. but to these recriminations i had a very natural answer to make and i made it.

"sir," said i, "no doubt you are ignorant of the discussions which have taken place concerning you in america and europe. you do not know that divers accidents, caused by collisions with your submarine machine, have excited public feeling in the two continents. i omit the hypotheses without number by which it was sought to explain the inexplicable phenomenon of which you alone possess the secret. but you must understand that, in pursuing you over the high seas of the pacific, the abraham lincoln believed itself to be chasing some powerful sea-monster, of which it was necessary to rid the ocean at any price." a half-smile curled the lips of the commander: then, in a calmer tone— "m. aronnax," he replied, "dare you affirm that your frigate would not as soon have pursued and cannonaded a submarine boat as a monster?" this question embarrassed me, for certainly captain farragut might not have hesitated. he might have thought it his duty to destroy a contrivance of this kind, as he would a gigantic narwhal. "you understand then, sir," continued the stranger, "that i have the right to treat you as enemies?" i answered nothing, purposely. for what good would it be to discuss such a proposition, when force could destroy the best arguments? "i have hesitated some time," continued the commander; "nothing obliged me to show you hospitality. if i chose to separate myself from you, i should have no interest in seeing you again; i could place you upon the deck of this vessel which has served you as a refuge, i could sink beneath the waters, and forget that you had ever existed. would not that be my right?" "it might be the right of a savage," i answered, "but not that of a civilised man." "professor," replied the commander, quickly, "i am not what you call a civilised man! i have done with society entirely, for reasons which i alone have the right of appreciating. i do not therefore obey its laws, and i desire you never to allude to them before me again!" this was said plainly. a flash of anger and disdain kindled in the eyes of the unknown, and i had a glimpse of

4. Semantic Security

(a)$E_1(k, m) := E(k, m)||\text{parity}(m)$
The following encryption function is NOT SEMANTICALLY SECURE.

Proof: Constructing an attack:

- Adversary A chooses two messages with different parities. For example, m_0 (even parity, so $\text{parity}(m_0) = 0$) and m_1 (odd parity, so $\text{parity}(m_1) = 1$).
- A sends m_0 and m_1 to the Challenger.
- The Challenger picks a random bit $b \in \{0, 1\}$, computes $c_b = E(k, m_b)||\text{parity}(m_b)$, and sends c_b back to A.
- A receives c_b and reads the last bit (the parity bit).
- If the last bit is 0, A outputs $b' = 0$. If the last bit is 1, A outputs $b' = 1$.

Since the last bit perfectly determines which message was encrypted, A always guesses correctly ($\Pr[b' = b] = 1$). Therefore, E_1 is not semantically secure.

(b)$E_2(k, m) := \text{reverse}(E(k, m))$
The following encryption function is SEMANTICALLY SECURE.

Proof: Suppose there exists an adversary A that breaks the semantic security of E_2. We construct adversary B to break the semantic security of E:

- B begins the game with the E-Challenger.
- A (who has played E_2 game) chooses two messages m_0 and m_1 and gives them to B.
- B forwards m_0 and m_1 to the E-Challenger.
- The E-Challenger randomly selects $b \in \{0, 1\}$ and returns the challenge ciphertext $c = E(k, m_b)$ to B.
- B computes $c' = \text{reverse}(c)$ and gives c' to A.
- Notice that $c' = \text{reverse}(E(k, m_b)) = E_2(k, m_b)$.
- B perfectly simulates the E_2 game for A. He outputs a guess b' and because A has broken the semantic security of E_2, $b' = b$ with 100% certainty.
- B outputs the exact same guess b', thus breaking the semantic security of E.

Thus, by Law of Contrapositive, E_2 is semantically secure.

(c) $E_3(k, m) := E(k, \text{reverse}(m))$
The following encryption function is SEMANTICALLY SECURE.

Proof: Suppose there exists an adversary A that breaks the semantic security of E_3. We construct adversary B to break the semantic security of E:

- B begins the game with the E-Challenger.
- A (who has played the E_3 game) chooses two messages m_0 and m_1 and gives them to B.
- B computes $m_0' = \text{reverse}(m_0)$ and $m_1' = \text{reverse}(m_1)$.
- B sends m_0' and m_1' to the E-Challenger.
- The E-Challenger randomly selects $b \in \{0, 1\}$ and returns the challenge ciphertext $c = E(k, m_b')$ to B.
- Notice that $c = E(k, \text{reverse}(m_b))$, which is exactly $E_3(k, m_b)$. B forwards c to A.
- B perfectly simulates the E_3 game for A. He outputs a guess b' and because A has broken the semantic security of E_3, $b' = b$ with 100% certainty.
- B outputs the exact same guess b', thus breaking the semantic security of E.

Thus, by Law of Contrapositive, E_3 is semantically secure.

Part 2 Questions

Q1. Master Key, Input Block, and Output Block Sizes

AES-128:
Master Key: 128 bits (16 bytes)
Input Block: 128 bits (16 bytes)
Output Block: 128 bits (16 bytes)

AES-192:
Master Key: 192 bits (24 bytes)
Input Block: 128 bits (16 bytes)
Output Block: 128 bits (16 bytes)

AES-256:
Master Key: 256 bits (32 bytes)
Input Block: 128 bits (16 bytes)
Output Block: 128 bits(16 bytes)

(Note: The block size is always 128 bits for all AES variants; only the key size varies.)

Q12. Need for the Initial AddRoundKey Step

If AES did not start with an initial AddRoundKey operation, the first round would start with SubBytes and ShiftRows before any secret key material is introduced. Since these substitution and permutation operations are completely public, and key-independent, an adversary knowing the plaintext could compute their results directly. This renders those initial operations cryptographically useless for security.

Adding the first round key at the very beginning ensures that every subsequent state and intermediate operation is immediately dependent on the secret key.

Q13. Problem with Padding Strategy in AES-CBC

The Problem: The padding strategy that is used, pads the plaintext with 0x00 and unpads it by stripping trailing 0x00 bytes. If the original plaintext naturally ends with one or more 0x00 bytes, the unpadding operation cannot distinguish them from the padding bytes and will incorrectly strip them, resulting in data loss.

The Solution: Use PKCS#7 Padding (RFC 5652).

- The number of padding bytes added is $V = 16 - (L \ mod \ 16)$, where L is the message length in bytes.
- Each padding byte is set to the value V. If the message is already a multiple of 16 bytes, a full block of 16 padding bytes containing 0x10 (16 in decimal) is added.

- Since every padded message contains at least 1 padding byte, and the value of the last byte indicates exactly how many padding bytes were added, the receiver can always correctly identify and strip the exact padding without ambiguity.

Q14. Tag forgery in AES-CBC Encryption

Yes, an adversary can forge a tag Tag_f for a message m_f without querying it from Alice. Let's look how this can be done:

1. Adversary queries Alice.
   - Query a single-block message m_1 to get $Tag_1 = E_k(m_1)$.
   - Query a single-block message m_2 to get $Tag_2 = E_k(m_2)$.
2. Constructing a message for which tag can be forged.
   - Construct a two-block message $m_f = m_1 \mid\mid (m_2 \oplus Tag_1)$.
   - Apply AES-CBC encryption on m_f with IV = 0x00.
     - Block 1: $c_1 = E_k(m_1 \oplus IV) = E_k(m_1) = Tag_1$.
     - Block 2: $c_2 = E_k((m_2 \oplus Tag_1) \oplus c_1) = E_k(m_2 \oplus Tag_1 \oplus Tag_1) = E_k(m_2) = Tag_2$.
   - The final ciphertext block is Tag_2. Therefore, $Tag_f = Tag_2$.
   - This is a forged tag, as we have not queried m_f from Alice.